QA ATELIER · DOCUMENT 2 OF 3

Test Case Definitions

Catalog, detailed definitions, and authoring conventions for RWA test cases

Author	Tomas Xavier Santos
Project	QA Atelier (tom-xs/qa-atelier)
AUT	Cypress Real World App
Date	2026-08-02
Version	1.1

Contents

1. Conventions

- Test cases carry sequential IDs (**TC-XXX**) and are managed as GitHub Issues from the test-case template, labeled `type: test-case` plus framework, app, and priority labels.
- Every test case states: requirement ID, priority, type, preconditions, steps, expected result, and test data.
- Automated tests follow **AAA** (Arrange–Act–Assert), Page Object Model for UI, `data-test` selectors, explicit waiting only, and full independence between tests.
- Names read as `[action] [expected result]`.

2. Detailed Test Cases — Existing

2.1 TC-002 — Create a payment transaction

Requirement	REQ-TX-001
Priority / Type	P1 / Functional
Framework	Cypress
Issue	qa-atelier #6 · Status: documented, not yet automated

Preconditions

- RWA is running at http://localhost:3000
- User Heath93 exists (seeded by default)

Steps

1. Log in and navigate to the dashboard
2. Click “New Transaction”
3. Search for another user
4. Select the first result
5. Enter an amount and description
6. Click “Pay”

Expected result

- A success alert is shown
- The transaction appears in the feed
- Sender balance decreases, receiver balance increases by the amount

2.2 TC-003 — Request money from another user

Requirement	REQ-TX-002
Priority / Type	P1 / Functional
Framework	Cypress
Issue	qa-atelier #7 · Status: documented, not yet automated

Preconditions

- RWA is running at http://localhost:3000
- Logged in as Heath93

Steps

1. Log in and navigate to the dashboard
2. Click “New Transaction”
3. Search for another user and select them
4. Enter an amount and description
5. Click “Request” instead of “Pay”

Expected result

- A success alert is shown
- The transaction appears in the feed with “requested” status
- The recipient sees a pending request in their notifications

2.3 TC-004 — User cannot log in with invalid credentials

Requirement	REQ-AUTH-002
Priority / Type	P0 / Smoke, Negative
Framework	Cypress
Issue	qa-atelier #8 · Status: partially automated — error-message assertion exists in `web/cypress/e2e/rwa/auth.cy.ts`; remaining: stays-on-`/signin` and no-cookie assertions

Preconditions

- RWA is running at http://localhost:3000

Steps

1. Navigate to http://localhost:3000
2. Enter username: Heath93
3. Enter an incorrect password: wrongpassword123
4. Click the sign in button

Expected result

- User remains on the login page
- An error message is displayed indicating invalid credentials
- No authentication token (session cookie) is stored

Test data: username `Heath93`, password `wrongpassword123`

2.4 TC-005 — RWA API auth collection + CI environment

Requirements	REQ-AUTH-001, REQ-AUTH-002
Priority / Type	P1 / API Contract
Framework	Postman / Newman
Issue	qa-atelier #12 · Status: documented, not yet automated

Collection `rwa-auth.postman_collection.json`:

Request	Assertions
POST /login (valid)	200; `user.id` non-empty string; `user.username` matches; `connect.sid` cookie present (`pm.cookies.has`); chain `userId` into environment
POST /login (bad password)	401; error message in body
GET /users	200 with session cookie (jar handles it); `results` is an array
GET /users/{{userId}}	200; profile matches logged-in user

Environment `ci.postman_environment.json`: `baseUrl=http://localhost:3001`, `username=Heath93`, `password` initial value `{{RWA_PASS}}` (real value only in current value / CI secret), empty `userId`.

Done when: collection and environment exported to `api/`; `bash api/newman/run-all.sh` green locally and in CI.

Scope note: the login response-time budget (< 500 ms) is intentionally **not** asserted here — it is owned by TC-020, so the budget check lives in exactly one case.

2.6 TC-021 — Client-side login validation blocks empty submission

Requirements	REQ-AUTH-001, REQ-AUTH-002
Priority / Type	P2 / Functional — client-side validation
Framework	Cypress
Issue	qa-atelier #31 · Status: automated

Preconditions: RWA at http://localhost:3000; no input entered yet.

Steps: open the sign-in page; verify no error/helper text before interaction; click **Sign In** with both fields empty.

Expected result: no error shown by default; submission blocked (no request reaches `POST /login`); helper text appears under the empty required fields.

Automation: covered by `blocks login with empty fields` and `doesn't display error message by default` in `web/cypress/e2e/rwa/auth.cy.ts`. Known defect in the current code: `.find("username-helper-text")` is not a valid CSS selector — should be `.find("#username-helper-text")`; fix during the TC-tagging refactor.

2.7 TC-022 — Log in with valid credentials via the UI

Requirement	REQ-AUTH-001
Priority / Type	P1 / Functional — Smoke, positive path
Framework	Cypress
Issue	qa-atelier #30 · Status: automated

Preconditions: RWA at http://localhost:3000; seeded user `Heath93` / `s3cret`; credentials via `CYPRESS_RWA_USER` / `CYPRESS_RWA_PASS`.

Steps: open the sign-in page; enter valid credentials; click **Sign In**.

Expected result: redirect to `/`; authenticated UI renders (notifications indicator in the top nav); `connect.sid` cookie set.

Automation: covered by `logs in successfully with valid credentials` in `web/cypress/e2e/rwa/auth.cy.ts`. Complements the ID-less Vitest auth suite (§2.5), which covers REQ-AUTH-001 at the API layer.

2.5 Vitest auth suite — automated

Requirements	REQ-AUTH-001, REQ-AUTH-002, REQ-AUTH-005
Priority / Type	P0 / API Contract

Framework	Vitest (`api/tests/rwa/auth.test.ts`)
Status	Automated, green in CI

Test	Assertions
POST /login returns the user and a session cookie	`user.id` non-empty string; `user.username` equals the login user; cookie matches `connect.sid=`
POST /login returns 401 for wrong password	status 401
Session cookie grants access to protected endpoints	`GET /users` → 200 with cookie, 401 without

Credentials from `RWA_USER` / `RWA_PASS` env vars with `||` fallback to public seed credentials (missing CI secrets expand to empty strings, which `??` would not catch).

3. Planned Test Cases

ID	Title	Requirement	Type	Priority	Target framework
TC-006	Log out invalidates the session	REQ-AUTH-003	Functional	P1	Cypress
TC-007	Sign up creates a new account	REQ-AUTH-004	Functional	P1	Cypress
TC-008	Session persists across reload	REQ-AUTH-005	Functional	P1	Playwright
TC-009	Link a new bank account	REQ-BANK-001	Functional	P1	Cypress
TC-010	Bank accounts list shows linked accounts	REQ-BANK-002	Functional	P1	Postman / Newman
TC-011	Feeds filter Mine / Friends / Public correctly	REQ-TX-003	Functional	P1	Cypress
TC-012	Recipient accepts a money request	REQ-TX-004	Functional	P1	Cypress
TC-013	Recipient rejects a money request	REQ-TX-004	Negative	P2	Cypress

TC-014	Payment above balance is rejected	REQ-TX-006	Negative	P1	Vitest (API)
TC-015	Like and comment on a transaction	REQ-TX-005	Functional	P2	Cypress
TC-016	Notification appears for received request	REQ-NOTIF-001	Functional	P1	Cypress
TC-017	User search returns matching users	REQ-USER-001	Functional	P2	Vitest (API)
TC-018	Update account settings persists	REQ-USER-003	Functional	P2	Playwright
TC-019	API schema contract: `/transactions` response shape	REQ-TX-001	API Contract	P1	Postman / Newman
TC-020	Response-time budget: login under 500 ms	REQ-AUTH-001	Non-functional	P2	Postman / Newman

Coverage gaps (no test case yet, all P2): REQ-USER-002 (view public profile), REQ-BANK-003 (delete bank account), REQ-NOTIF-002 (dismiss notifications).

4. Authoring Workflow for New Test Cases

1. Open an issue from the **Test case** template; it auto-labels `type: test-case`.
2. Fill requirement ID, priority, type, preconditions, numbered steps, exact expected result, test data table.
3. Add framework, app, and priority labels; place it on the board in **Backlog**.
4. Move through Exploration → In Scripting → In Review (PR open) → Done (merged, `status: automated`).
5. Update the RTM (Document 1) in the same PR that merges the automation.